CBSE CLASS XII – COMPUTER SCIENCE

Python Programming – Practical

1) PROGRAM:Write a Program on Stack Implementation

```
# PROGRAM: Stack Operations Implementation using List
# Checks if the stack is completely empty
def isEmpty(stk) :
    if stk == [] :
        return True
    else :
        return False
# Logic to insert an element into the stack
def Push(stk, item) :
    stk.append(item)
    top = len(stk) - 1
# Logic to delete and return the top element
def Pop(stk) :
    if isEmpty(stk) :
        return "Underflow"
    else :
        item = stk.pop()
        if len(stk) == 0 :
            top = None
        else :
            top = len(stk) - 1
        return item
# Returns the topmost element without removing it
def Peek(stk) :
    if isEmpty(stk) :
        return "Underflow"
    else :
        top = len(stk) - 1
        return stk[top]
# Displays stack elements from top to bottom
def Display(stk) :
    if isEmpty(stk) :
        print("Stack empty")
    else :
        top = len(stk) - 1
        print(stk[top], "<--top")
        for a in range(top-1, -1, -1) :
            print(stk[a])
```

```
Stack = []
top = None                    # initially stack is empty

while True :
    print("STACK OPERATIONS")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display stack")
    print("5. Exit")
    ch = int(input("Enter your choice (1-5) :"))
    if ch == 1 :
        item = int(input("Enter item :"))
        Push(Stack, item)
    elif ch == 2 :
        item = Pop(Stack)
        if item == "Underflow" :
            print("Underflow! Stack is empty!")
        else :
            print("Popped item is", item)
    elif ch == 3 :
        item = Peek(Stack)
        if item == "Underflow" :
            print("Underflow! Stack is empty!")
        else :
            print("Topmost item is", item)
    elif ch == 4 :
        Display(Stack)
    elif ch == 5 :
        break
    else :
        print("Invalid choice!")


OUTPUT:

STACK OPERATIONS
1. Push
2. Pop
3. Peek
4. Display stack
5. Exit
Enter your choice (1-5) :1
Enter item :2
```

2) PROGRAM: Write a Python program to create a binary file named "July.dat" using the pickle module. Accept the roll number, name and marks in Mathematics, Computer Science subjects for n students.

```
# PROGRAM: enter marks in Mathematics, Computer Science and AI subjects
import pickle
# Open binary file in write-binary mode
f = open("July.dat", "wb")
n = int(input("Enter total no. of students: "))

for i in range(n):
    roll = int(input("Enter roll no. of student: "))
    name = input("Enter student's name: ")
    math = float(input("Enter Maths marks: "))
    cs = float(input("Enter CS marks: "))
    record = [roll, name, math, cs]
    pickle.dump(record, f)
f.close()
print("Record entry over")

OUTPUT:
Enter total no. of students: 2
Enter roll no. of student: 1
Enter student's name: abrar
Enter Maths marks: 90
Enter CS marks: 99
Enter roll no. of student: 2
Enter student's name: zehra
Enter Maths marks: 89
Enter CS marks: 96
Record entry over
```

3) PROGRAM:Write a Python program to search for a student record with given roll number in a binary file named "marks.dat" using the pickle module. Display the complete record if the roll number is found; otherwise, display "Roll no. not found".

```
# PROGRAM: Search a Student Record in Binary File by Roll Number
import pickle
# Opening the binary file in read-binary mode
f = open("marks.dat", "rb")
r = int(input("Enter the roll no. to be searched: "))
found = False
```

```
try:
    while True:
        record = pickle.load(f) # Reading records one by one

        if record[0] == r:    #Checking if the rno matches the search key
            print("Record:", record)
            found = True
            break
except EOFError:        # Exception Handling: Reached end of file safely
    pass

if found == False:
    print("Roll no. not found")

f.close()
```

```
OUTPUT:
Enter the roll no. to be searched: 2
Record: [2, 'zehra', 258.0, 86.0]
```

4) PROGRAM: A college administrator, Atul, maintains student course enrollment records. Each record includes Roll_No, Student_Name, Course, Fees.
Write Python functions to:

I. Input enrollment details and append them to a binary file.
II. Update the Fees of all students enrolled in the "Data Science" course to 75000.

```
Program:

# PROGRAM: College Course Enrollment Management using Binary File
import pickle

def AddEnrollment():
    # Input student details and append to binary file
    Roll = int(input("Enter Roll No: "))
    Name = input("Enter Student Name: ")
    Course = input("Enter Course: ")
    Fees = int(input("Enter Fees: "))
    rec = [Roll, Name, Course, Fees]
    #Open in append-binary mode to add records without overwriting
    f = open("Enrollments.dat", "ab")
    pickle.dump(rec, f)
    f.close()
    print("Record added successfully.")
```

```
# Update fees for Data Science course students
def UpdateFees():
    updated = False
    records = []
# Read all records and modify target course data
    f = open("Enrollments.dat", "rb")
    try:
        while True:
            rec = pickle.load(f)
            if rec[2] == "Data Science":
                rec[3] = 75000
                updated = True
            records.append(rec)
    except EOFError:
        pass
    f.close()
#Rewrite the modified records back to the file
    f = open("Enrollments.dat", "wb")
    for rec in records:
        pickle.dump(rec, f)
    f.close()

    if updated:
        print("Fees updated for Data Science students.")
    else:
        print("No student enrolled in Data Science found.")
AddEnrollment()  #call functions
UpdateFees()

OUTPUT:
Enter Roll No: 101
Enter Student Name: abrar
Enter Course: Data Science
Enter Fees: 60000
Record added successfully.
Fees updated for Data Science students.
```

5) PROGRAM: Write a Python program to delete a student record from a binary file named "student.dat" based on the roll number entered by the user. Use a temporary file named "temp.dat" to store the records that are not deleted. Display an appropriate message if the record is deleted or if the roll number is not found. Also handle the case when the file does not exist.

ANSWER

```
# PROGRAM: Delete a Student Record from Binary File using a Temporary File
import pickle
import os

def delete_record():
    rollno = int(input("Enter roll no. to delete: "))
    found = False
# Open original binary file to read and a temporary file to write
    try:
        f = open("student.dat", "rb")
        temp = open("temp.dat", "wb")

        try:
            while True:
                rec = pickle.load(f)
                if rec[0] == rollno: #compare rollno to be deleted
                    found = True
                else:
                    pickle.dump(rec, temp)

# Exception Handling: Reached end of file safely
        except EOFError:
            pass

        f.close()
        temp.close()
# Logic: Update the main database file if a deletion occurred
        if found:
            os.replace("temp.dat", "student.dat")
            print("Record deleted successfully")
        else:
            os.remove("temp.dat")
            print("Record not found")
#Handle case when the target database doesn't exist
    except FileNotFoundError:
        print("File not found")
delete_record()
```

Output:

```
Enter roll no. to delete: 102
Record deleted successfully
```

6) PROGRAM: Kabir runs a vehicle rental agency and maintains a CSV file named Rentals.csv. The file stores daily rental details with the following columns:
Vehicle_ID, Vehicle_Type, Hours_Rented, Rate_Per_Hour
Create the following user-defined functions:
I. AddRental() – To accept rental details from the user and append them to Rentals.csv.
II. CalculateTotalRent() – To calculate and return the total rent earned using the formula:
Total Rent = Hours_Rented × Rate_Per_Hour

```
# PROGRAM: Vehicle Rental Record Management using CSV File
import csv
def AddRental():
    # Input rental details from the user
    Vehicle_ID = input("Enter Vehicle ID: ")
    Vehicle_Type = input("Enter Vehicle Type: ")
    Hours_Rented = input("Enter Hours Rented: ")
    Rate_Per_Hour = input("Enter Rate Per Hour: ")

    # Open CSV file in append mode with newline flag to prevent empty rows
    f=open("Rentals.csv", "a", newline="")
    w = csv.writer(f)
    # Logic: Write fields as a single row list into the CSV
    w.writerow([Vehicle_ID, Vehicle_Type, Hours_Rented,
Rate_Per_Hour])
    f.close()
    print("Rental record added successfully.")

def CalculateTotalRent():
    total = 0
    # Logic: Open CSV file in read-only mode to calculate values

    f = open("Rentals.csv", "r")
    r = csv.reader(f)
    #Traverse rows, convert strings to integers, and aggregate
the total
    for row in r:
        hours = int(row[2])
        rate = int(row[3])
        total += hours * rate
    f.close()
    return total
```

```
    f.close()  #close file
AddRental()    #call function call
print(CalculateTotalRent())

OUTPUT:
Enter Vehicle ID: 1
Enter Vehicle Type: bike
Enter Hours Rented: 3
Enter Rate Per Hour: 500
Rental record added successfully.
1500
```

7) PROGRAM: Write a Python function avg_len() that adds up the lengths of all the words in a list and then prints the average (mean) length.

```
# PROGRAM: Calculate the average length of words in a list
def avg_len(words):
    total = 0
    count = 0
# Iterates through each word in the provided list
    for w in words:
        total = total+len(w)
        count = count+1

    return total / count  # Returns the calculated mathematical average length
my_words = ["computer", "science", "python", "data"]  #define list
print("Average word length:", avg_len(my_words))  #function call

OUTPUT:
Average word length: 6.25
```

---

8) PROGRAM: Write a Python function capitalize_words(words) that takes a list of strings words and returns a new list with each word capitalize. Example: If the input is ["python", "computer", "science"] The output should be ["Python", "Computer", "Science"]."

```
# PROGRAM: Capitalize the first letter of each word in a list
words= ["python", "computer", "science"]
def capitalize_words(words):
    result = []
# Iterates through each word in the list
```

```
# Capitalizes the first letter and appends it to the new list
    for word in words:
        result.append(word.capitalize())

    return result   # return result
new_list= capitalize_words(words)   #FUNCTION CALL
print(new_list)

OUTPUT:
['Python', 'Computer', 'Science']
```

9) PROGRAM: Write a Python function that displays the number of times the word "Python" appears in a text file named "demo.txt

```
# PROGRAM: Count occurrences of the word 'Python' in a text file
def count_python():
    count = 0
    # Opening file and reading its content as a list of words
    f=open("demo.txt","r")
    text = f.read()
    words = text.split()
# Iterating through words to find case-insensitive matches
    for word in words:
        if word.lower() == "python":
            count += 1
    print("The word Python appears", count, "times.")
    f.close() # Closing file
count_python()

OUTPUT:
The word Python appears 4 times.
```

10) PROGRAM: Write and call a Python function to read History.txt and display all lines that contain a four-digit year (e.g., 1999, 2024 etc.)

For example:
History.txt
India became independent in 1947.
Palestine was declared a state in 1988.
Palestine gained UN observer status in 2012.
The republic Congo

```
# PROGRAM: Display lines containing a 4-digit year from a text file
def DisplayYearLines():
```

```
    f = open("History.txt", "r")  # Opening file in read-only mode
# Iterating through the file line by line
    for line in f:
        words = line.split()
# Checking each word to identify a 4-digit number (year)
        for w in words:
            if w.isdigit() and len(w) == 4:
                print(line)
                break

    f.close()  # Closing the file

DisplayYearLines()  #function call

OUTPUT:
India became independent in 1947.
Palestine was declared a state in 1988.
Palestine gained UN observer status in 2012.
```

11) PROGRAM: Write a Python program using the random module to generate and display 5 random integers between 1 and 50.

```
# PROGRAM: Generate and print five random integers between 1 and 50
import random  # Importing the random module
# Loop to execute the block exactly five times
for i in range(5):
    print(random.randint(1, 50))    # Generates and prints a random
integer

OUTPUT:
20
48
46
32
36
```

12) PROGRAM: Create a MySQL table named Student with the following fields: Roll_No, Name, Class and Marks. Insert 5 suitable records into the table.

```
CREATE TABLE Student
```

```
(
    Roll_No INT,
    Name VARCHAR(30),
    Class VARCHAR(10),
    Marks INT
);

INSERT INTO Student VALUES
(101, 'Abrar', 'XI', 78),
(102, 'Fatima', 'XI', 65),
(103, 'Zehra', 'XII', 92),
(104, 'Abbas', 'XII', 35),
(105, 'Mahdi', 'XII', 88);

SELECT * FROM Student;
```

Output

```
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 101     | Abrar  | XI    | 78    |
| 102     | Fatima | XI    | 65    |
| 103     | Zehra  | XII   | 92    |
| 104     | Abbas  | XII   | 35    |
| 105     | Mahdi  | XII   | 88    |
+---------+--------+-------+-------+
------------------------
```

---

13) PROGRAM: The Student table contains the following records:

Roll_No	Name	Class	Marks
101	Abrar	XI	78
102	Fatima	XI	65
103	Zehra	XII	92
104	Abbas	XII	35
105	Mahdi	XII	88

Using the above Student table, perform the following ALTER TABLE operations:

(a) Add a new attribute City.

(b) Drop the attribute City.

Program:
(a) Add a new attribute City:

```
ALTER TABLE Student
ADD City VARCHAR(20);
SELECT * FROM Student;
```

Output:

The new `City` column will be added. Since you haven't entered any city values, they will be `NULL`.

```
+---------+--------+-------+-------+------+
| Roll_No | Name   | Class | Marks | City |
+---------+--------+-------+-------+------+
| 101     | Abrar  | XI    | 78    | NULL |
| 102     | Fatima | XI    | 65    | NULL |
| 103     | Zehra  | XII   | 92    | NULL |
| 104     | Abbas  | XII   | 35    | NULL |
| 105     | Mahdi  | XII   | 88    | NULL |
+---------+--------+-------+-------+------+
```

(b) Drop the attribute City:

```
ALTER TABLE Student
DROP City;
SELECT * FROM Student;
```

Output: The `City` column is completely removed:

```
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 101     | Abrar  | XI    | 78    |
| 102     | Fatima | XI    | 65    |
| 103     | Zehra  | XII   | 92    |
| 104     | Abbas  | XII   | 35    |
| 105     | Mahdi  | XII   | 88    |
+---------+--------+-------+-------+
-----------------------
```

14) PROGRAM:Using the Student table, write MySQL queries to: (a) Change the marks of the student having Roll_No = 105 to 99. (b) Delete the student record having Roll_No = 101.

Roll_No	Name	Class	Marks
101	Abrar	XI	78
102	Fatima	XI	65
103	Zehra	XII	92
104	Abbas	XII	35
105	Mahdi	XII	88

```
Program:
(a) Change the marks of Roll_No 105 to 99:
UPDATE Student
SET Marks = 99
WHERE Roll_No = 105;
SELECT * FROM Student;
OUTPUT:
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 101     | Abrar  | XI    | 78    |
| 102     | Fatima | XI    | 65    |
| 103     | Zehra  | XII   | 92    |
| 104     | Abbas  | XII   | 35    |
| 105     | Mahdi  | XII   | 99    |
+---------+--------+-------+-------+

(b) Delete the student having Roll_No 101:
DELETE FROM Student
WHERE Roll_No = 101;
SELECT * FROM Student;
OUTPUT:
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 102     | Fatima | XI    | 65    |
| 103     | Zehra  | XII   | 92    |
| 104     | Abbas  | XII   | 35    |
| 105     | Mahdi  | XII   | 99    |
+---------+--------+-------+-------+
```

---

15) PROGRAM:Using the Student table, write MySQL queries to:
(a) Display all student records in ascending order of Marks.
(b) Display all student records in descending order of Marks.

Roll_No	Name	Class	Marks
101	Abrar	XI	78
102	Fatima	XI	65
103	Zehra	XII	92
104	Abbas	XII	35
105	Mahdi	XII	88

Program:

(a) Display all student records in ascending order of Marks:

```
SELECT * FROM Student
ORDER BY Marks ASC;
```

OUTPUT:

```
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 104     | Abbas  | XII   | 35    |
| 102     | Fatima | XI    | 65    |
| 101     | Abrar  | XI    | 78    |
| 105     | Mahdi  | XII   | 88    |
| 103     | Zehra  | XII   | 92    |
+---------+--------+-------+-------+
```

(b) Display all student records in descending order of Marks:

```
SELECT * FROM Student
ORDER BY Marks DESC;
```

OUTPUT:

```
+---------+--------+-------+-------+
| Roll_No | Name   | Class | Marks |
+---------+--------+-------+-------+
| 103     | Zehra  | XII   | 92    |
| 105     | Mahdi  | XII   | 88    |
| 101     | Abrar  | XI    | 78    |
| 102     | Fatima | XI    | 65    |
| 104     | Abbas  | XII   | 35    |
+---------+--------+-------+-------+
------------------------
```

---

16) PROGRAM: MYSQL – Cartesian Product

Table 1: Student

Roll_No	Name
101	Abrar
102	Sana

Table 2: Subject

Subject_ID	Subject
1	CS
2	Maths

PROGRAM:

```
SELECT * FROM Student, Subject;
```

Output: Cartesian Product

```
+---------+-------+------------+---------+
| Roll_No | Name  | Subject_ID | Subject |
+---------+-------+------------+---------+
| 101     | Abrar | 1          | CS      |
| 101     | Abrar | 2          | Maths   |
| 102     | Sana  | 1          | CS      |
| 102     | Sana  | 2          | Maths   |
+---------+-------+------------+---------+
```

17) PROGRAM: MySQL database named LibraryDB has a table book_records which contains the following attributes:

- **Book_ID: ID of the book (Integer)**
- **Book_Title: Title of the book (String)**
- **Author: Author of the book (String)**
- **Copies: Number of copies available (Integer)**

Consider the following details to establish Python-MySQL connectivity:

- **Username: librarian**
- **Password: lib@2025**
- **Host: localhost**

Write a Python-MySQL program to change the Book_Title to 'Data Structures in Python' for the record whose Book_ID is 109 in the book_records table

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="librarian",
    password="lib@2025",
    database="LibraryDB"
)

cur = con.cursor()

q = "UPDATE book_records SET Book_Title='Data Structures in Python'
WHERE Book_ID=109"

cur.execute(q)
con.commit()

print("Book title updated successfully.")

cur.close()
con.close()
```

18) PROGRAM: Consider the table TECH_EVENT_REG as given below:

reg_id	participant_name	event_name	grade	reg_fee
501	Aarav	Hackathon	11	200
502	Abrar	RoboRace	12	150
503	Abbas	WebDev Challenge	12	250
504	Zainab	Hackathon	11	200
505	Ali	CodeSprint	9	100
506	Fatima	RoboRace	10	150

Write the following queries with outputs:

IQ. To display the number of participants in each event where the number of participants is greater than 1.

Ans)

SELECT event_name, count(*) AS total_participants FROM tech_event_reg GROUP BY event_name HAVING count(*) > 1;

OUTPUT:

event_name	total_participants
Hackathon	2
RoboRace	2

II. To display the distinct event names available in the Tech Event.

Ans) SELECT DISTINCT event_name FROM tech_event_reg;

OUTPUT:

event_name
Hackathon
RoboRace
WebDev Challenge
CodeSprint

III. To display details of participants whose names contain the letters 'na'.

Ans) SELECT * FROM tech_event_reg WHERE participant_name LIKE '%na%';

OUTPUT:

reg_id	participant_name	event_name	grade	reg_fee
504	Zainab	Hackathon	11	200

IV. To display all records of TECH_EVENT_REG sorted by participant_name in descending order.

Ans) SELECT * FROM tech_event_reg ORDER BY participant_name DESC;

OUTPUT:

reg_id	participant_name	event_name	grade	reg_fee
504	Zainab	Hackathon	11	200
506	Fatima	RoboRace	10	150
505	Ali	CodeSprint	9	100
502	Abrar	RoboRace	12	150
503	Abbas	WebDev Challenge	12	250
501	Aarav	Hackathon	11	200